Design Document

ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

Course: COEN838 — Advanced Software Engineering
Student: Idris Muhammad Abubakar
Matric No: P25EGCP9003

---

1. Architectural Overview

1.1 Architectural Style

ABU-SSO is a **modular monolith** structured as a **layered architecture**, organised around **feature routers** rather than technical layers.

Why modular monolith (and not microservices from day one)?

Consideration	Modular Monolith	Microservices
Team size	✅ 1 developer	❌ Needs 3+ teams
Deployment complexity	✅ Single process	❌ Container orchestration
Learning curve	✅ Simple	❌ High
Growth path	✅ Services can be extracted	✅ Native
Consistency	✅ Single transaction	❌ Saga needed

The modular monolith was chosen because the project is built by a single developer within a course timeframe, but the **service boundaries are already drawn** so any router (`auth`, `wallet`, `id_card`, `admin`) could be extracted into its own microservice without rewriting business logic.

A real distributed layer was subsequently added — three independent mock services (Portal, LMS, Library) that trust ABU-SSO as their identity provider. See Section 8.

1.2 Layer Responsibilities

Rule enforced: each layer only talks to the one directly below it. Routers never touch the database directly — they call services.

1.3 Inter-Component Communication

- **Client → Server:** HTTP/1.1 with JSON bodies (or HTML for pages).
- **Server → Third-party services:** synchronous REST; the JWT is forwarded in `Authorization: Bearer`.
- **Service → IdP (for token verification):** synchronous REST call to `/auth/me`.
- **Style chosen:** synchronous request-response, because authentication decisions must be immediate.

Future asynchronous events (e.g. "notify user on wallet top-up") would use a message broker — out of scope for this prototype.

2. Component Design

2.1 Authentication Component

File	Responsibility
`app/routers/auth.py`	HTTP endpoints: `/auth/register`, `/auth/login`, `/auth/me`
`app/services/user_service.py`	`create_user`, `authenticate_user`, lookup helpers
`app/utils/security.py`	`hash_password`, `verify_password` (bcrypt)
`app/utils/jwt.py`	`create_access_token`, `decode_token`, `get_current_user` dependency
`app/utils/rbac.py`	`require_role(*roles)` — dependency factory for role-gated endpoints
`app/schemas/user.py`	Pydantic schemas: `UserCreate`, `UserRead`, `UserLogin`, `Token`

Key design decision: `get_current_user` is a FastAPI dependency. Any protected route declares `current_user: User = Depends(get_current_user)` — the authentication logic lives in exactly one place.

2.2 Wallet Component

File	Responsibility
`app/routers/wallet.py`	Endpoints: `/wallet`, `/wallet/topup`, `/wallet/pay`, `/wallet/transactions`
`app/services/wallet_service.py`	`get_or_create_wallet`, `top_up`, `pay`, `list_transactions`
`app/models/wallet.py`	`Wallet` + `Transaction` ORM models
`app/schemas/wallet.py`	`WalletRead`, `TransactionRead`, `TopUpRequest`, `PayRequest`

Key design decision: every wallet operation is **atomic** — the balance update and the transaction insert happen in a single `db.commit()`. If the balance is insufficient, `pay()` raises before touching the DB.

2.3 Digital ID Card Component

File	Responsibility
`app/routers/id_card.py`	Endpoint `/id-card`
`app/services/qr_service.py`	`build_id_card_payload`, `generate_qr_data_url`
`app/schemas/id_card.py`	`IdCardResponse`

Key design decision: the QR code encodes only `{id, matric_no, role}` — no personal data — reducing the risk of information disclosure if a card is photographed.

2.4 Admin Component (RBAC)

File	Responsibility
`app/routers/admin.py`	Endpoints: `/admin/users`, `/admin/users/{id}/role`, `/admin/users/{id}/active`, `/admin/wallet/topup`, `/admin/stats`
`app/services/admin_service.py`	User listing, role changes, activation, admin top-ups, stats aggregation
`app/schemas/admin.py`	`UserAdminView`, `AdminTopUpRequest`, `AdminRoleUpdateRequest`, `AdminStats`
`app/utils/rbac.py`	`require_role("admin")` — 403 for non-admins

Key design decision: the entire admin router declares `dependencies=[Depends(require_role("admin"))]` — every endpoint in the router is automatically protected. Non-admin callers receive HTTP 403 Forbidden.

2.5 Presentation Component

File	Responsibility
`app/routers/pages.py`	Serves `/login`, `/dashboard`, `/admin`
`app/templates/base.html`, `login.html`, `dashboard.html`, `admin.html`	Jinja2 templates
`app/static/css/style.css`	Green campus theme
`app/static/js/app.js`	Shared JS; page-specific logic inline in templates

The dashboard calls `/auth/me`, `/wallet`, `/wallet/transactions`, and `/id-card` using the JWT stored in `localStorage`. The admin page additionally calls `/admin/*`.

---

3. Data Design

3.1 Entity Relationship Diagram

See `docs/diagrams/class_model.png`.

3.2 Table Definitions

`users`

Column	Type	Constraint
id	VARCHAR(36)	PRIMARY KEY
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED
password_hash	VARCHAR(255)	NOT NULL

Column	Type	Constraint
full_name	VARCHAR(255)	NULL
role	VARCHAR(50)	NOT NULL, default 'student'
matric_no	VARCHAR(50)	NULL, INDEXED
is_active	BOOLEAN	NOT NULL, default TRUE
created_at	DATETIME	NOT NULL

`wallets`

Column	Type	Constraint
id	VARCHAR(36)	PRIMARY KEY
user_id	VARCHAR(36)	FOREIGN KEY → users.id, UNIQUE
balance	NUMERIC(10,2)	NOT NULL, default 0
created_at	DATETIME	NOT NULL

`transactions`

Column	Type	Constraint
id	VARCHAR(36)	PRIMARY KEY
wallet_id	VARCHAR(36)	FOREIGN KEY → wallets.id
amount	NUMERIC(10,2)	NOT NULL
type	VARCHAR(20)	NOT NULL, `credit` or `debit`
description	VARCHAR(255)	NULL
created_at	DATETIME	NOT NULL

3.3 Design Decisions

- **UUIDs instead of integers** for primary keys — safe to expose in URLs, work identically in SQLite and PostgreSQL.
- `Numeric(10,2)` **for money** — avoids floating-point rounding errors.
- **Wallet created lazily** — users who never use the wallet don't consume a row.
- **Cascade delete** — deleting a wallet removes its transactions automatically.

---

4. Sequence Design — Payment Flow

See `docs/diagrams/sequence_login.png` for the login sequence. Below is the wallet payment flow in text form:

Client (browser) │ │ POST /wallet/pay {amount: 250} │ Authorization: Bearer ▼ Router (wallet.py: pay_from_wallet) │ ├─ Depends(get_current_user) ──► JWT validated, User loaded │ ├─ Depends(get_db) ──────────────► DB session opened │ ├─ wallet_service.get_or_create_wallet(db, user) │ └─ wallet_service.pay(db, wallet, 250) │ ├─ if amount <= 0: raise ValueError ├─ if balance < amount: raise ValueError │ ├─ wallet.balance -= 250 ├─ db.add(Transaction(type='debit', amount=250)) ├─ db.commit() │ └─ return Transaction │ ▼ Client receives 201 Created + transaction JSON

---

5. Resilience & Scalability

5.1 Resilience

- **Fail securely:** any auth failure returns HTTP 401 with a generic message — no info leak about whether the user exists.
- **Authorization failure:** non-admin callers receive 403 with a clear "Requires role: admin" message.
- **Transaction atomicity:** wallet balance and transaction record are committed together; a crash between the two cannot occur.
- **Stateless auth:** if any server instance dies, the user's token continues to work on another instance.

5.2 Scalability

- **Horizontal scaling:** the app is stateless (JWT), so multiple instances can run behind a load balancer.
- **Database growth:** SQLite is single-writer; at production scale, change `DATABASE_URL` to PostgreSQL — no code changes.
- **Read replicas:** future work can add PostgreSQL read replicas for `/auth/me` reads.
- **Mock services are independently scalable** — each runs on its own port and can be replicated.

5.3 Known Limitations

- No rate limiting yet (planned via `slowapi`).
- No refresh tokens — users must log in every 15 minutes.
- No async task queue — everything is synchronous in the request cycle.
- Mock services use URL-based token passing (production would use cookies or BFF).

---

6. Design Patterns Applied

Pattern	Where	Why
Repository / Service Layer	`app/services/*.py`	Business logic separated from HTTP
Dependency Injection	FastAPI `Depends()` for DB and current user	Testable; single source of truth
Schema / DTO	Pydantic `*Create`, `*Read` classes	Never leak ORM internals to API
Singleton	`settings = Settings()` in `config.py`	One configuration source

Pattern	Where	Why
Factory	`create_engine`, `sessionmaker` in `database.py`	Configurable DB backend
Factory (dependency)	`require_role(*roles)` in `rbac.py`	Reusable role-gating
Middleware / Decorator	`get_current_user` dependency	Cross-cutting auth for all protected routes
Layered Architecture	Routers / Services / Models	Clear responsibilities
Service-Oriented (mock services)	`mock-services/*/main.py`	Independent deployable units

7. Technology Choices — Justification

Choice	Alternative	Why we chose ours
FastAPI	Flask / Django	Async-native, auto-generated OpenAPI, type-driven
SQLAlchemy	Raw SQL / Django ORM	Portable between SQLite and PostgreSQL
SQLite (dev)	PostgreSQL from day 1	Zero setup for a beginner; migrate later
JWT (HS256)	Server-side sessions	Stateless; ideal for SSO across services
bcrypt	SHA-256	Designed for password storage; salted; slow by design
RBAC middleware	Hard-coded role checks	Reusable, testable, single source of truth
Jinja2 + vanilla JS	React / Vue	Simpler; sufficient for a three-page UI
pytest	unittest	Cleaner syntax, better fixtures
GitHub Actions	Jenkins	Free, integrated with GitHub
Bandit	Manual review	Automated SAST — evidence for security rubric
Render + Neon	Single-provider hosting	Free tier stability; separation of compute and storage

8. Distributed SSO Architecture (Mock Services)

To demonstrate that ABU-SSO is genuinely a **distributed, service-oriented system** (not merely a login page), three independent mock services were built that trust ABU-SSO as their identity provider.

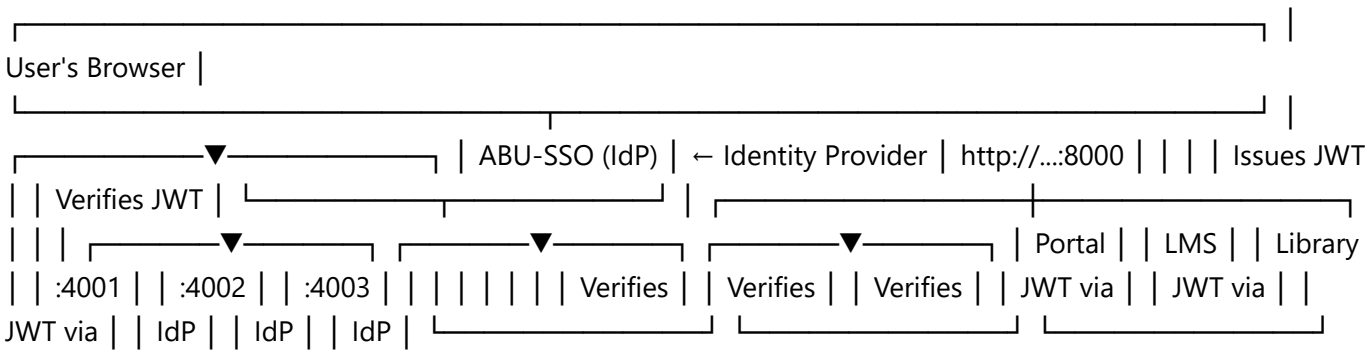
8.1 Why Three Services?

The original problem statement (COEN838 Question 7) says:

> *"Students currently juggle multiple credentials: the central student portal, the LMS (Moodle), the Wi-Fi portal, and physical ID cards."*

Building three separate services that **independently verify the SSO token** proves the core claim: **one login → many services**.

8.2 Service Topology



8.3 Design Decisions

Decision	Choice	Justification
Service boundary	Each service is a **standalone FastAPI app** in its own process	Independence — one can crash without affecting the others
Data ownership	No service has a user table	Single source of truth remains the IdP; services are stateless verifiers
Token verification	HTTP call to `IdP/auth/me`	Simpler than sharing the JWT secret; models how OIDC UserInfo works
Token transport	URL query param `?token=<JWT>`	Simplest cross-origin demo; production would use HttpOnly cookies or BFF pattern
Coupling	Loose — services know only the IdP's base URL	Swap or scale IdP without touching services
Failure mode	If IdP is down, service returns HTTP 503	Fail-secure: never serve data without verification

8.4 Inter-Service Communication Patterns

Pattern used: Synchronous REST with token-based trust.

- Each service performs `GET {IDP_URL}/auth/me` with the JWT in the `Authorization` header.
- Response is cached in-memory for the request duration (not persisted).
- Latency cost: one additional HTTP round-trip (~5–10 ms locally).

Alternative patterns considered:

Pattern	Why we didn't choose it (for this prototype)
Shared JWT secret (each service verifies signature)	Requires distributing secret; harder to rotate; couples services to algorithm choice

Pattern	Why we didn't choose it (for this prototype)
RS256 public-key verification	Better for production; overkill for a demo where IdP is co-located
Event-based (Kafka/RabbitMQ)	Correct for async workloads; unnecessary for a synchronous read
OIDC Discovery + JWKS	Production-grade; requires HTTPS and DNS setup beyond demo scope

8.5 Scalability & Resilience Implications

- **Horizontal scale-out of services** — services are stateless, so N instances can run behind a load balancer.
- **IdP is the bottleneck** — every request from every service calls `/auth/me`. In production, this would be mitigated by:
    1. Caching verified tokens at the service for their remaining TTL (up to 15 min).
    2. Using RS256 with JWKS so services verify locally without calling the IdP.
- **Failure isolation** — if Library service dies, Portal and LMS remain fully functional.
- **Fail-secure** — if the IdP is unreachable, services return 503, never serve data unauthenticated.

8.6 How It Maps to the Rubric

Rubric line	How the mock services satisfy it
"distributed/service-oriented solution"	Three independent services on distinct ports, each a separate OS process
"inter-component communication pattern"	Synchronous REST; token passed via HTTP header; documented above
"scalability"	Stateless services, horizontally scalable; IdP bottleneck documented with mitigations
"resilience"	Fail-secure when IdP unreachable; failure isolation across services

8.7 Files

File	Purpose
`mock-services/portal/main.py`	Standalone FastAPI app on port 4001
`mock-services/lms/main.py`	Standalone FastAPI app on port 4002
`mock-services/library/main.py`	Standalone FastAPI app on port 4003
`mock-services/portal/templates/service.html`	Shared template (copied to all three)

8.8 Demo Flow

1. User logs in once at ABU-SSO (`http://127.0.0.1:8000/login`).
2. Dashboard renders four service cards.

3. Clicking **Student Portal** opens `http://127.0.0.1:4001/?token=<JWT>`.
4. The Portal service:
    - Extracts the token from the query string.
    - Calls `GET http://127.0.0.1:8000/auth/me` with the token.
    - If the IdP returns 200, renders the Portal page with the user's profile.
    - If not, returns HTTP 401.
5. Same flow for LMS and Library — **no second password prompt in any of them**.

This demonstrates the **single sign-on** guarantee end-to-end.

End of Design Document.